Programação Avançada em Java
Trabalho Prático 4

Introdução

Objetivo: À semelhança dos projetos anteriores, este projeto deverá ser construído sobre a solução desenvolvida para o Projeto 3. A persistência de dados deverá ser mantida do lado do servidor, com recurso a uma base de dados PostgreSQL. Devem igualmente ser utilizados JPA e endpoints REST para a gestão dos respetivos dados. O principal objetivo do projeto é **adaptar o frontend** de forma a ser utilizado react.js. Pretende-se assim que desenvolvam as vossas capacidades nesta Framework de Javascript, compreendendo as principais vantagens e desvantagens em relação a Javascript puro. As funcionalidades a considerar são exatamente as mesmas do Projeto 3.

Data de Entrega: 1 de Abril de 2026

Grupos: o projeto é realizado **individualmente.**

Arquitetura do Sistema a Desenvolver

A arquitetura a adotar deverá ser semelhante à já implementada no Projeto 3, com exceção do frontend. Pretende-se assim que, ao invés de ser utilizado Javascript puro, seja desenvolvido um frontend em reavt.js. O facto de o react.js implementar um sistema de gestão de estados, permitir utilizar JSX e usar um DOM virtual faz com que esta seja uma ferramenta mais eficiente para a construção de soluções mais complexas. De forma resumida, a Figura 1 representa uma visão geral da arquitetura a ser implementada.

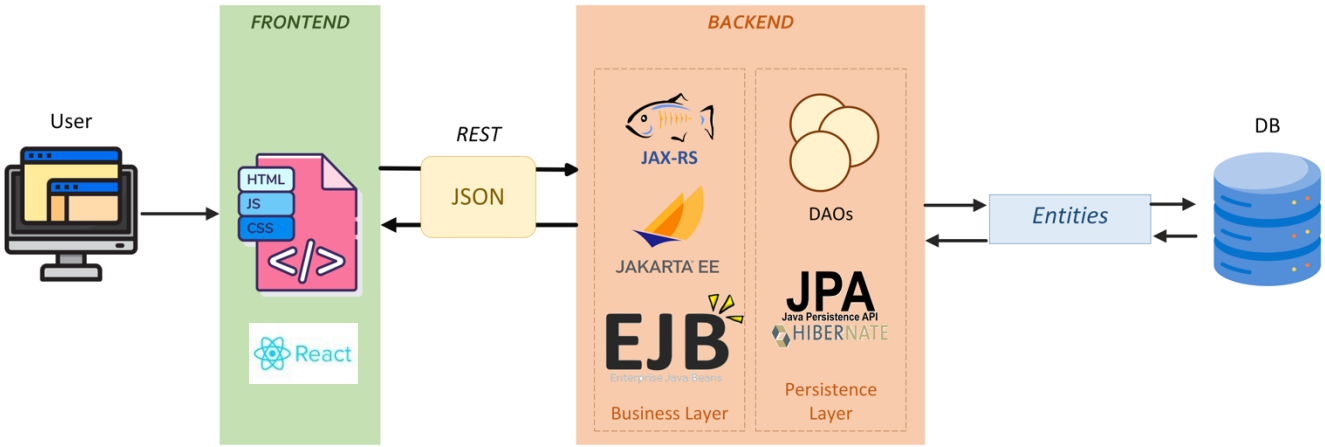

Figura 1. Arquitetura do sistema a implementar

Neste sentido, o **Backend** corresponde a um projeto **Maven** que deverá interagir com uma base de dados *PostgreSQL* usando *Jakarta Persistance* com *Hibernate.* Não são expectáveis alterações ao modelo de dados desenvolvido para o **Projeto 3**, ficando à vossa liberdade quaisquer ajustes. Assim, o **backend** é responsável por expor uma *REST API* que deverá ser consumida pelo frontend, sendo que todos os dados enviados e recebidos devem estar em formato *JSON*.

Por fim, o **novo frontend** deverá ser construído em **react.js,** não sendo um requisito fazer alterações ao aspeto visual existente. Pretende-se acima de tudo que a interface exponha todas as funcionalidades desenvolvidas para o **Projeto 3**, utilizando boas práticas de *react.js.*

REST API

Os endpoints da REST API utilizados no Projeto 3 podem ser alterados, ficando ao vosso critério adicionar, alterar ou remover endpoints. É expectável que a API siga as boas práticas de uma RESTful API:

- Devolver os status codes adequados para cada um dos pedidos
- Promover uma comunicação de dados em formato JSON
- Utilizar nomes ao invés de verbos nos endpoints, sendo o método do pedido HTTP (POST, GET, ...) responsável por indicar a ação a executar. Por exemplo:
    - Efetuar um GET ao endpoint `/leads/` para obter todos os leads ao invés de `GET /allLeads/`. Neste cenário, o método GET mostra claramente que queremos obter o recurso `/leads/`. Caso queiramos obter o lead com o ID igual a 1, podemos utilizar `GET /leads/1`
    - Efetuar um POST ao endpoint `/leads/` para inserir um novo lead, ao invés de `POST /insertLead/`

Outro aspeto importante a ter em conta está relacionado com a filtragem de recursos. Por exemplo, quando se pretende filtrar tarefas por utilizador, esta operação deverá ser realizada na base de dados PostgreSQL, ao invés do frontend. Este é um ponto muito

importante do ponto de vista de performance, sendo a prática mais correta para a gestão de dados. Notem o seguinte exemplo:

Operação: Obter todos os leads do utilizador A e com o estado B

Fluxo:

1. Frontend faz pedido à API: `GET /leads/?utilizador=A&estado=B`
2. No backend é executada uma e uma só query SQL, onde a clausula "WHERE" permite filtrar por *utilizador* e *estado.*
3. O resultado é devolvido ao frontend, sendo este apenas responsável por apresentar ao utilizador a informação. **Não é executada nenhuma filtragem no frontend**, uma vez que os dados já foram filtrados no backend.

O exemplo acima deverá servir de inspiração e guia para a forma como deverá ser obtida informação e apresentada ao utilizador.

Frontend – react.js

Pretende-se que o **frontend** existente para o Projeto 3 seja adaptado para que seja utilizado react.js. Para a gestão de estados, recomenda-se a utilização do Zustand, embora possam optar por alternativas como Mobx ou Redux. Independentemente da vossa escolha, é importante que promovam uma organização lógica e fundamentada para a vossas *stores*. Por exemplo, fará sentido existir apenas uma store com todas as variáveis? Provavelmente não. No entanto, ter uma store apenas dedicada para a gestão de leads poderá promover uma melhor organização.

FAQ

1- Posso manter o meu backend tal como está e apenas implementar o novo frontend em react.js?

Em teoria sim, uma vez que o objetivo principal do projeto é implementarem um frontend em react.js. Contudo, é importante melhorar o backend, alterar endpoints ou mesmo adicionar novos caso considerem necessários. É importante, por exemplo, garantir que a filtragem/pesquisa e ordenação de informação é realizada no backend (base de dados), em vez de nofrontend.

2- Qual a biblioteca de gestão de estados que devo utilizar? Zustand, Mobx ou Redux?

Recomendamos o Zustand pela sua simplicidade e facilidade de integração. Nas aulas será maioritariamente explorado o Zustand. Contudo, o Mobx e o Redux são alternativas válidas, sendo em regra mais utilizados em aplicações de maior complexidade. Sintam-se à vontade para escolher e, acima de tudo, aprender aquela que vos desperta mais curiosidade.

3- É expectável que o aspeto visual da aplicação seja alterado?

Não. O objetivo é adaptar o frontend de forma a ser utilizado react.js. Assim, a identidade visual da aplicação pode manter-se, ficando ao vosso critério fazerem as melhorias que considerem pertinentes e que vos deixem mais satisfeitos com o resultado.

4- No meu gestor de estados (Mobx, React ou Zustand), devo ter apenas uma store ou várias?

Ter apenas uma *store* sería estranho, dada a complexidade do projeto. O ideal é tentarmos pensar nas várias entidades que temos no sistema e espelhá-las nas nossas stores. Por exemplo, temos um utilizador e leads. Pode fazer sentido ter uma *store* para utilizadores e outra para leads.

Entrega do projeto e avaliação

Entregas: os projetos devem ser guardados num **repositório *Git***. Só será considerado para avaliação o código do **último *commit*** feito na data indicada na introdução deste enunciado.

Defesa: As defesas terão uma duração de **25 minutos**, sendo necessário uma inscrição prévia na plataforma InforEstudante, na página da disciplina. Durante a defesa, todo o projeto deve estar a funcionar e deve ser executado sem erros. Se durante a defesa o código apresentar diferenças face ao entregue, será **descontado 25%** da nota. Também é importante garantir que os alunos cheguem à defesa preparados (i.e., código aberto no IDE, web browser com a aplicação pronta a mostrar). Caso contrário será aplicada uma **penalização de 10%**.

Nota importante: qualquer componente implementada, mas não demonstrada na defesa recebe **50% de penalização**.